Formalizing a Price-Time Priority Matching Engine in TLA+: A Specification Case Study

Abstract

We report on the formalization and bounded exhaustive exploration of a nontrivial financial matching-engine specification using TLA+ and the TLC model checker. The specification covers a full order-type suite—LIMIT, MARKET, MARKET-TO-LIMIT, STOP_LIMIT, STOP_MARKET—augmented with iceberg orders, four self-trade prevention (STP) policies, post-only, Fill-Or-Kill, minimum-quantity, cancel, and amend semantics. Within the explored configurations TLC found one definitive FIFO invariant violation caused by a missing timestamp refresh on triggered stop orders, and surfaced a second specification gap—an iceberg order stranding reachable when STP DECREMENT zeros a resting order's visible quantity without triggering the iceberg reload path. Both issues are absent from the prose specification; neither was caught in prior manual review of that document. This paper describes the model, the invariants, the counterexamples, the fixes, and the concrete lessons about feature-composition hazards in exchange matching semantics.

1 Introduction

Matching-engine correctness is commercially important and surprisingly difficult to state precisely. Production engines combine a large number of interacting features—order types, time-in-force policies, self-trade prevention, hidden quantity, stop triggers, cascading fills—each specified correctly in isolation, but capable of producing unexpected emergent behavior when combined. Prose specifications resolve the easy cases and leave the hard ones to implementor judgment.

Formal methods offer a sharper alternative: write the semantics as an executable state machine and enumerate its reachable states. This paper is an experience report on doing exactly that for a matching engine specification. We make no claim that the TLA+ model constitutes a proof of correctness for any implementation. What we claim, more modestly, is that bounded exhaustive exploration of the model's state space found two specification bugs that were not visible in the prose document and that are operationally significant.

The contributions of this paper are:

1. A TLA+ formalization of a feature-rich price-time-priority matching engine specification.
2. A suite of invariants covering book consistency, queue fairness, and order-type safety conditions.
3. A concrete TLC counterexample showing that triggered stop orders can violate FIFO if activation does not refresh queue priority.
4. Identification of an additional semantic flaw involving self-trade-prevention decrement and iceberg visibility management.
5. Minimal specification amendments and lessons about composing advanced order semantics.

The paper is organized as follows. Section 2 describes the matching-engine semantics under study. Section 3 presents the TLA+ model and the scope of the bounded exploration. Section 4 enumerates the invariants checked. Section 5 presents the findings in detail, with counterexample traces. Section 6 shows the fixes applied. Section 7 draws lessons about feature-composition hazards. Section 8 states the limitations and reproducibility conditions. Section 9 concludes.

2 Matching-Engine Semantics Under Study

2.1 Specification Source

The object of study is an internal prose specification, `matching-engine-formal-spec.md` version 1.2.0, describing a generic asset-agnostic order matching engine with price-time priority (FIFO). The specification defines primitive domains, order well-formedness constraints, the order book structure, a core matching algorithm, post-match disposition, six order types, four STP policies, iceberg reload semantics, stop triggering with cascading evaluation, cancel, and amend.

The specification is 900 lines of structured pseudocode. It was written with care—well-formedness constraints are enumerated, invariants are stated formally, and edge cases such as MTL price derivation and FOK dry-run semantics are treated explicitly. It is not a toy specification.

2.2 Order Types and Features

The engine supports five order types:

- **LIMIT**: rests at a specified price; fills at that price or better.
- **MARKET**: fills at any available price; never rests on the book.
- **MARKET-TO-LIMIT (MTL)**: aggressively matches at any price; locks to the first execution price and rests any remainder as a LIMIT order.
- **STOP_LIMIT / STOP_MARKET**: dormant until a trigger condition on the last trade price is satisfied, then converted to LIMIT or MARKET respectively and re-entered into the matching pipeline.

Order modifiers include:

- **Iceberg** (`displayQty`): only $visibleQty \leq displayQty$ is shown to the market; after the visible slice is consumed, the order reloads with a fresh visible slice and loses time priority.
- **Post-Only**: the order must not aggress; rejected (under the REJECT policy) if it would cross the spread.
- **TimeInForce**: GTC, GTD, IOC, FOK, DAY.
- **MinQty**: requires a minimum fill on first execution; the constraint is cleared after the initial threshold is met, allowing the remainder to rest.
- **Self-Trade Prevention (STP)**: four policies—CANCEL_NEWEST, CANCEL_OLDEST, CANCEL_BOTH, DECREMENT—govern what happens when an incoming order would trade against a resting order in the same STP group. The incoming order's policy governs.

2.3 Core Invariants Stated in the Specification

The prose specification states fourteen invariants that must hold after every `PROCESS` call completes (§13 of the spec). The FIFO invariant (INV-7/INV-8 in the spec) is particularly relevant here:

> For every price level, consecutive orders satisfy: $o_1.timestamp < o_2.timestamp$.

This is the formal statement of price-time priority. A violation means an order with a later arrival time has higher queue position than one that arrived earlier—the queue is out of order.

3 TLA+ Model and Bounded Exploration

3.1 Model Structure

The TLA+ specification (`MatchingEngine.tla`, approximately 850 lines) is a direct translation of the prose specification. The state consists of eight variables:

```
bidQ           : PRICES → Seq(Order)   (* per-price bid queues *)
askQ           : PRICES → Seq(Order)   (* per-price ask queues *)
stops          : Set(Order)             (* dormant stop orders *)
lastTradePrice : PRICES \cup {NULL}     (* last execution price *)
postOnlyOK     : Bool                   (* no post-only aggressor
   trade *)
stpOK          : Bool                   (* no same-group trade *)
nextId         : Nat                    (* next order ID *)
clock          : Nat                    (* logical clock *)
```

The model has four actions: `SubmitOrder`, `CancelOrder`, `AmendOrder`, and `TimeAdvance`. `SubmitOrder` nondeterministically selects all valid order parameters and invokes the full processing pipeline. TLC's exhaustive exploration therefore considers every valid combination of order type, side, price, quantity, TIF, STP policy, iceberg flag, post-only flag, and minimum quantity, within the configured bounds.

3.2 Processing Pipeline

The pipeline operator `ProcessOrder` implements all phases of the prose specification's `PROCESS` procedure, including recursive operators for the matching loop (`DoMatch`) and stop-trigger cascade (`ProcessCascade`, `ProcessTriggeredStops`). Mutual recursion between these operators is required to model cascading stop triggers correctly: a triggered stop may generate trades, which may trigger further stops.

A `fuel` parameter on `DoMatch` bounds recursion depth for TLC termination safety. Under the small quantity bounds used in model checking, this bound is never reached in practice.

3.3 Scope of Exploration

The model is parameterized by `PRICES`, `MAX_QTY`, `MAX_ORDERS`, and `MAX_CLOCK`. Configurations explored are shown in Table 1. All configurations include the full order-type suite, all four STP policies, icebergs, post-only, FOK, MinQty, and DAY expiry. The "With Amend" configuration adds the `AmendOrder` action.

Table 1: Model checking configurations and statistics.

Configuration	Orders	Qty	Prices	Amend	States Gen.	Distinct	Runtime	Result
Tiny	2	1	{1,2}	No	2,979,719	1,123,333	20 s	Pass
Small	2	2	{1,2}	No	11,528,343	6,037,674	1 m 42 s	Pass
Medium	2	2	{1,2,3}	No	36,700,016	21,261,901	5 m 34 s	Pass
With Amend	2	2	{1,2}	Yes	25,209,607	9,104,902	3 m 33 s	Pass
3-order	3	2	{1,2}	No	>45 M	>26 M	>10 m	*partial*

All "Pass" results above are post-fix. Prior to fixing the stop-timestamp bug, the Small configuration produced a counterexample on the `FIFOWithinLevel` invariant within the first 11.5 million states. The 3-order run was interrupted before state-space exhaustion; "partial" means no violation was found in the explored states, not clean passage under all 3-order behaviors.

3.4 What the Model Does Not Cover

- The model uses a single STP group identifier (`"G1"`). Multiple independent groups are not explored.
- GTD expiry is modeled via `TimeAdvance` but without wall-clock constraints.
- Event ordering (INV-10) and market-data publication are not modeled.
- The model checks specification consistency, not implementation correctness.

4 Invariants Checked

All invariants are checked as TLA+ safety properties (`INVARIANT` declarations in the TLC configuration). TLC verifies that each invariant holds in every reachable state within the explored configurations.

Table 2: Invariants checked.

ID	TLA+ Operator	Description	Status
INV-1/2	`NoEmptyLevels`	Every price key with a non-empty queue has length > 0	Trivially true
INV-3/4	`BookUncrossed`	$bestBid < bestAsk$, or either side empty	Pass
INV-5/6	`NoGhosts`	Every resting order has $remainingQty > 0$	Pass
INV-6/7	`StatusConsistency`	Resting orders have status NEW or PARTIAL	Pass
INV-7/8	`FIFOWithinLevel`	Consecutive orders have strictly increasing timestamps	**Violated → fixed → Pass**
INV-8/9	`NoRestingMarkets`	MARKET orders never appear on the book	Pass
INV-9	(construction)	Execution price equals resting order's price	True by construction
INV-11	`PostOnlyGuarantee`	No trade where the aggressor was post-only	Pass
INV-12	`STPGuarantee`	No trade between orders in the same STP group	Pass
INV-13	`NoRestingMTL`	MTL orders are converted before resting	Pass
INV-14	`NoRestingMinQty`	$minQty$ is cleared before an order rests	Pass

5 Findings and Counterexamples

5.1 Finding 1: FIFO Violation via Missing Timestamp on Triggered Stops

Severity. High—directly violates INV-7/8.

Root cause location. §10.1 `EVALUATE_STOPS` interaction with §6.1 `INSERT`.

The Mechanism

The prose specification's stop-trigger procedure (§10.1) converts a dormant stop order to its base type (LIMIT or MARKET) and re-enters it into the `PROCESS` pipeline. The stop order's `timestamp` field is not updated at trigger time. It retains the timestamp assigned when the order was originally submitted and placed into dormancy.

When the triggered LIMIT order does not fully fill and rests on the book via `DISPOSE` (§5.2), it is appended to the back of the appropriate price-level queue per §6.1—giving it the lowest priority at that level. However, `FIFOWithinLevel` requires that the newly appended order have a timestamp strictly greater than all orders already in the queue. Because orders already in the queue arrived after the stop was submitted—they carry later timestamps—the stop's original timestamp is numerically earlier. The queue is now out of order.

Counterexample Trace

TLC produced the following minimal counterexample. Times shown are logical clock values.

```
Step 1: Submit SELL LIMIT  qty=1, price=1      (id=1, ts=1)
        -> Rests on ask:  askQ[1] = [order1(ts=1)]

Step 2: Submit BUY STOP_LIMIT  stop=1, price=1, qty=1  (id=2, ts=2)
        -> lastTradePrice = NULL -> does not trigger
        -> Placed in stops = {order2(ts=2)}

Step 3: Submit BUY LIMIT  qty=2, price=1       (id=3, ts=3)
        -> Matches askQ[1]: fills order1 fully (fillQty=1)
        -> Trade: price=1, aggressor=3, passive=1
        -> lastTradePrice = 1
        -> order3: remainingQty=1, rests on bid
           bidQ[1] = [order3(ts=3)]
        -> Stop cascade: lastTradePrice(1) >= stopPrice(1)
           -> order2 triggers, converts to BUY LIMIT
           -> Timestamp NOT updated: ts=2 (STALE)
           -> Ask side empty, no fill
           -> Dispose: order2 rests on bid
              bidQ[1] = [order3(ts=3), order2(ts=2)]

        FIFOWithinLevel check:
            bidQ[1][1].timestamp = 3
            bidQ[1][2].timestamp = 2
            3 < 2 ?  FALSE  ->  INVARIANT VIOLATED
```

Figure 1: TLC counterexample trace for the FIFO violation. The triggered stop order (id=2) retains its original timestamp (ts=2) and is appended behind an order with a later timestamp (ts=3), producing a non-monotonic queue.

Why Manual Review Missed This

The prose specification treats stop triggering and order insertion as separate sections (§10.1 and §6.1 respectively). Each section is locally correct: §10.1 correctly describes conversion, and §6.1 correctly describes insertion. The interaction—that converted stops carry stale timestamps—is invisible when reading either section in isolation. A reviewer must simultaneously hold the conversion path, the insertion path, and the FIFO invariant in mind to see the problem. TLC holds all three simultaneously by construction.

5.2 Finding 2: Iceberg Stranding Under STP DECREMENT

Severity. Medium—creates a reachable book state with invalid order geometry.

Root cause location. §8.3 DECREMENT case; missing interaction with §7.5 iceberg reload.

The Mechanism

The STP DECREMENT policy reduces both the incoming order's *remainingQty* and the resting order's *visibleQty* and *remainingQty* by min(*incoming.remainingQty*, *resting.visibleQty*), without generating a trade. The iceberg reload logic (§7.5 step 3) specifies:

> When *visibleQty* = 0 and *remainingQty* > 0, reload the visible slice, assign a new timestamp, and move the order to the back of the queue.

However, this reload is only specified inside the fill path. The DECREMENT action (§8.3) does not reference §7.5. There is no clause in §8.3 for the case where DECREMENT drives *visibleQty* to zero while *remainingQty* > 0.

The result is a resting iceberg order with *visibleQty* $= 0$ and *remainingQty* > 0. It cannot be matched—$fillQty = \min(incoming.remainingQty, 0) = 0$—and cannot reload because the reload trigger is only in the fill path. The order is stranded: it occupies a queue position, blocks no fills, and cannot be removed except by explicit cancel.

Illustrative Scenario

```
Resting iceberg: qty=2, displayQty=1, visibleQty=1,
                 remainingQty=2, stpGroup="G1"        (id=R)
Incoming:        qty=1, stpGroup="G1", stpPolicy=DECREMENT  (id=I)

DECREMENT:
    reduceQty = min(1, 1) = 1
    I.remainingQty = 0   -> I is cancelled
    R.visibleQty   = 0
    R.remainingQty = 1   -> R remains on book

Post-decrement state of R:
    visibleQty=0, remainingQty=1, displayQty=1
    -> Reload path not invoked
    -> R sits on book, invisible, unreachable by matching
```

Note on claim level. This scenario is reachable in the prose specification as written. However, the TLA+ model as checked incorporates the fix (iceberg reload after DECREMENT) because the stranded state would cause the model checker itself to loop: subsequent orders would find *visibleQty* $= 0$ and skip the order without decrementing fuel, creating an effective infinite loop in `DoMatch`. The bug is therefore presented as a specification gap—a path the spec silently omits—rather than as a TLC-witnessed invariant violation with a precise counterexample trace.

5.3 Design Observations (Not Demonstrated Bugs)

The following interactions are noted as potential specification ambiguities. They are not presented as verified bugs because no invariant violation was produced for them.

Observation 1—FOK + STP DECREMENT. The FOK pre-check counts non-conflicting visible quantity. During matching, DECREMENT can consume the incoming order's *remainingQty* without generating trades. The incoming order reaches *remainingQty* $= 0$ and is treated as cancelled, but with fewer trade fills than *qty*. Whether this constitutes a FOK violation depends on whether FOK is defined as "fully traded" or "*remainingQty* fully consumed." The spec does not address this interaction.

Observation 2—MinQty + STP DECREMENT. Analogously, the MinQty pre-check guarantees *filled_quantity* $\geq$ *minQty* (§5.4), but if DECREMENT consumes part of the incoming quantity before any fills occur, the traded quantity may fall below *minQty* even though the pre-check passed.

Observation 3—MTL + MinQty clearing. The spec clears *minQty* in two places: once in the MTL path (Phase 4) and once in the normal matching path (Phase 5a). This duplication

is currently correct but fragile—adding a new order type or routing path requires remembering to add a third clearing point.

6 Specification Fixes

Both confirmed issues were fixed in the TLA+ model. The fixes are minimal and directly address the root causes.

Table 3: Bug and fix summary.

ID	Location	Description	Fix	Invariant
BUG-1	§10.1 `EVALUATE_STOPS`	Triggered stop retains original timestamp; violates FIFO when resting	Assign `stop.timestamp = currentTimestamp()` before re-entering pipeline	INV-7/8
BUG-2	§8.3 DECREMENT	Iceberg *visibleQty* driven to 0 without reload; order stranded	After DECREMENT: if $visibleQty = 0 \wedge remainingQty > 0 \wedge displayQty \neq \perp$, reload visible slice, refresh timestamp, move to back	Spec gap

6.1 Fix for BUG-1 (Stop Timestamp)

The fix is a one-line addition to §10.1. In the TLA+ model, `ProcessTriggeredStops` applies the fix:

```
converted == [ConvertStop(s) EXCEPT !.timestamp = tm]
```

The corresponding prose amendment to §10.1:

```
-- After "Convert to base type":
stop.timestamp = currentTimestamp()   -- Assign trigger-time priority
PROCESS(stop, book)
```

This ensures that a triggered stop order receives a timestamp reflecting when it entered the live matching pipeline, not when it was originally submitted.

6.2 Fix for BUG-2 (Iceberg Reload After DECREMENT)

The fix adds a post-DECREMENT iceberg check to §8.3. In the TLA+ model:

```
ELSE IF newRest.visibleQty = 0 ∧ newRest.displayQty /= NULL THEN
    LET reloadQty == Min(newRest.displayQty, newRest.remainingQty)
        reloaded  == [newRest EXCEPT !.visibleQty = reloadQty,
                                     !.timestamp = tm,
                                     !.status = "PARTIAL"]
        sc == SetContra(side, bQ, aQ, bestP,
                        restTail \o ⟨reloaded⟩)
    IN DoMatch(newInc, sc.bQ, sc.aQ, tds, tm + 1, fuel - 1)
```

The corresponding prose amendment to §8.3, after the DECREMENT quantity update:

```
IF resting.visibleQty = 0 AND resting.remainingQty > 0
   AND resting.displayQty != null:
```

```
    resting.visibleQty = min(resting.displayQty, resting.remainingQty)
    resting.timestamp  = currentTimestamp()
    MOVE resting TO back OF level.orders
```

This mirrors the iceberg reload behavior specified in §7.5 for the normal fill path, extending it to the DECREMENT path for consistency.

7 Lessons Learned

7.1 Feature Composition Is Where Bugs Hide

Both findings share a structural pattern: each involved feature is correctly specified in isolation, but the specification omits a clause covering their interaction. Stop orders are correctly described; iceberg reload is correctly described; FIFO insertion is correctly described. None of these sections contains an error when read alone. The bug lives in the intersection: what happens to an iceberg order's timestamp when the mechanism that zeros its visible quantity is not the fill path but the STP DECREMENT path?

This is not a failure of careful writing—it is a structural limitation of prose specifications. Prose sections have natural scope boundaries. Interactions between features that span sections are systematically harder to see than behaviors within a single section.

A formal model collapses all features into a single operational semantics and evaluates invariants globally. The intersection cases are not special—they are just more states.

7.2 Logical Clocks Amplify Ordering Bugs

The FIFO invariant is stated in terms of timestamps. Because the specification uses a monotonically increasing logical clock, any state transition that fails to advance the clock before inserting an order can produce a stale timestamp that violates ordering. The stop-trigger bug is exactly this: the clock was not advanced before the triggered stop was assigned to the book.

Timestamp-based ordering invariants are fragile in the presence of features (like stop triggers) that re-enter orders into the pipeline from a dormant pool. Wherever an order exits a dormant state and re-enters an active queue, its timestamp must be refreshed. The fix is simple; the risk is that prose specifications do not enumerate every re-entry point.

7.3 Iceberg Reload Is Not Tied Tightly Enough to Its Trigger Condition

The iceberg reload semantics in §7.5 are described in terms of the fill path: "when *visibleQty* is fully consumed [by a fill]." The prose implicitly assumes that *visibleQty* can only reach zero through the fill path. STP DECREMENT provides a second path to zero *visibleQty* that the reload trigger does not anticipate.

A more robust specification would state the reload condition declaratively as an invariant on the resting order state:

> A resting order must not have $visibleQty = 0$ and $remainingQty > 0$.

Rather than specifying reload as a step triggered by the fill path, it would be specified as a consequence of this invariant: any action that could produce this state must be followed by a reload. This declarative framing would make the omission in DECREMENT immediately apparent.

7.4 Bounded Exploration Is Sufficient for Shallow Bugs

Both bugs in this report were found in the 2-order, 2-quantity, 2-price configuration (11.5 million states, 102 seconds). They did not require large configurations. Shallow bugs—those reachable within a small number of orders—are precisely the category that bounded exhaustive exploration handles well. The two-order FIFO counterexample requires exactly three orders (one resting ask, one stop, one incoming buyer) and is found quickly.

7.5 The Specification Artifact Has Independent Value

The TLA+ model serves as a precise, executable reference for the prose specification. Independently of the bugs found, it forces every semantic choice to be made explicit: what is the recursion structure of stop cascades? What exactly does "timestamp" mean for a triggered order? At what point does MTL price derivation occur relative to the cascade? These questions have definite answers in the model even when the prose is ambiguous.

8 Limitations and Reproducibility

8.1 Bounded Exhaustive Exploration, Not Proof

TLC performs bounded exhaustive state enumeration: it finds every state reachable from `Init` via the `Next` relation, within the parameter bounds. A clean run means no invariant violation was found within the explored configurations. It does not mean no violation exists for larger configurations, more orders, or more price levels. We do not claim the specification is correct in any unbounded sense.

Specifically:

- The 3-order partial run did not complete state-space exploration.
- No configuration with more than two distinct STP groups was checked.
- GTD time-based expiry was checked only via the simplified `TimeAdvance` action, not against wall-clock constraints.
- The event-ordering invariant (INV-10) is not formalized in the model.

8.2 Specification vs. Implementation

This work verifies the prose specification against itself. The TLA+ model is a formalization of the prose spec; invariant violations indicate inconsistencies or omissions in the spec. An implementation that correctly implements the (fixed) specification could still contain implementation bugs—off-by-one errors, concurrency hazards, data-structure corruption—that are outside the scope of this model.

8.3 Reproducibility

The complete artifacts are:

- `MatchingEngine.tla`—the TLA+ specification (∼850 lines)
- `MatchingEngine.cfg`—TLC configuration (default)
- `MatchingEngine_*.cfg`—per-configuration files for Table 1
- `matching-engine-formal-spec.md` v1.2.0—the prose specification
- `REPORT.md`—model checking output and bug descriptions

To reproduce the FIFO counterexample:

1. Revert the stop timestamp fix in `ProcessTriggeredStops`: replace `[ConvertStop(s) EXCEPT !.timestamp = tm]` with `ConvertStop(s)`.
2. Run TLC with the Small configuration (`MAX_ORDERS=2, MAX_QTY=2, PRICES={1,2}`, amend disabled).

3. TLC will find a violation of `FIFOWithinLevel` within the first several million states.

9 Conclusion

We formalized a 900-line prose matching-engine specification in TLA+ and ran bounded exhaustive exploration across several configurations totaling over 36 million distinct states. Within the explored configurations, TLC found one definitive specification bug—a FIFO ordering violation caused by a triggered stop order retaining its original submission timestamp—and surfaced one specification gap—the iceberg reload path is not invoked after STP DECREMENT zeros a resting iceberg's visible quantity.

The flagship finding is the FIFO violation. Its counterexample is a three-order sequence, operationally simple, not obviously related to stop orders when reading the FIFO invariant definition. Normal prose review of the specification did not find it. TLC found it in under two minutes.

The broader lesson is not that formal methods are infallible, but that they are complementary to prose review in a specific way: they are systematic about feature intersections. Stop triggers and FIFO ordering are described in different sections of the specification. A model checker has no sections. Both features are expressed in the same operational semantics, and violations that cross feature boundaries are found without requiring the reviewer to remember to check every pairing.

The counterexample and fixes are small and reproducible. That is the appropriate standard for an experience report: not "we proved the engine correct," but "here is a concrete bug, here is why it was not obvious, and here is the minimal fix."